PROJET: LE JEU 2048

06 AVRIL 2026

RAPPORT DE PROJET
Application Mobile Jeu 2048

Nom: BALDE Amadou Dioulde ET KABA Mohamed

Matière: Développement sous Smartphone

Année: 2025–2026

Encadrant: Mohammed Islam NAAS

Table des Matières :

1. Introduction

2. Présentation de l’application

3. Technologies utilisées

3.1 Kotlin

3.2 Jetpack Compose

4. Architecture

5. Fonctionnalités principales

6. Fonctionnalités avancées

6.1 Multi-joueurs

6.2 Défi quotidien

1. INTRODUCTION:

Dans un contexte où les technologies mobiles occupent une place centrale dans notre quotidien, le développement d'applications Android constitue un domaine essentiel de l'informatique moderne. Ce projet s'inscrit dans cette dynamique en proposant la conception et la réalisation d'une application mobile inspirée du célèbre jeu 2048, un jeu de réflexion alliant simplicité d'utilisation et complexité algorithmique.

L'objectif principal de ce travail est de mettre en pratique les concepts fondamentaux du développement mobile, en s'appuyant sur le langage Kotlin et sur l'outil Jetpack Compose pour la création d'interfaces utilisateurs modernes et interactives.

À travers ce projet, il s'agit non seulement de reproduire le fonctionnement du jeu 2048, mais également d'implémenter une architecture logicielle propre et maintenable, notamment en adoptant le modèle MVVM (Model – View – ViewModel).

Le développement de cette application a permis d'explorer plusieurs aspects clés tels que la gestion des événements utilisateur (gestes et interactions), la manipulation dynamique des données, la persistance de l'état de l'application, ainsi que l'optimisation de l'expérience utilisateur grâce à des animations et des fonctionnalités avancées. Le projet met également en évidence l'importance de la séparation des responsabilités, de la modularité du code et de l'utilisation de bonnes pratiques de programmation.

Ainsi, ce travail constitue une mise en œuvre concrète des connaissances acquises en développement mobile, tout en offrant une application fonctionnelle, interactive et évolutive. Il illustre également la capacité à concevoir une solution complète, allant de la réflexion initiale jusqu'à la réalisation technique et à la présentation du produit final.

2. L'objectif du projet était de concevoir une application mobile complète intégrant:

- ✓ Une interface utilisateur moderne
- ✓ Une logique algorithmique optimisée
- ✓ Un système de sauvegarde automatique
- ✓ Une base de données locale
- ✓ Plusieurs modes de jeu avancés
- ✓ Une personnalisation complète
- ✓ Un système de partage
- ✓ Un déploiement réel sur Play Store

Contrairement à une implémentation classique du jeu 2048, notre application propose une version enrichie avec plusieurs fonctionnalités avancées améliorant l'expérience utilisateur.

Actuellement, l'application est déployée sur Google Play Store en phase de test, permettant une validation réelle sur appareil mobile.

2.1 Présentation générale de l'application :

L'application développée est une implémentation moderne du jeu 2048 intégrant :

- Mode classique
- Sauvegarde automatique
- Classement local
- Statistiques utilisateur
- Thèmes personnalisables
- Multi-joueurs local
- Défi quotidien chronométré
- Tutoriel interactif
- Partage du score en image

Elle repose sur une architecture logicielle modulaire séparant :

- Interface graphique
- Logique métier
- Gestion des données
- Services système Android
- Cette séparation garantit :
- Maintenabilité
- Évolutivité
- Testabilité
- Performance

3. Technologies utilisées

3.1 Langage principal

L'application est développée en Kotlin, langage officiel Android recommandé par Google.

Avantage :

- Sécurité mémoire améliorée
- Syntaxe concise
- Compatibilité Java
- Support cortines

3.2 Interface graphique :

L'interface utilisateur est réalisée avec :

- Jetpack Compose
- Framework UI déclaratif moderne permettant :
- Mise à jour automatique des écrans
- Gestion simple des états
- Animations fluides
- Réduction du code XML

3.3 Architecture logicielle

L'application suit l'architecture :

- MVVM (Model View ViewModel)
- Elle sépare :
- Model de Gestion :
- Grille de jeu
- Score
- Règles
- Fusion des tuiles
- ViewModel
- Responsable :
- Logique applicative
- Communication UI ↔ moteur
- Sauvegarde automatique
- Gestion des modes
- View

Interface Compose :

- GameScreen
- SettingsScreen
- MultiplayerScreen
- ChallengeScreen
- ScoresScreen
- Cette architecture améliore la lisibilité du projet.

4. Architecture globale de l'application

- ✓ L'application est structurée en plusieurs couches :
- ✓ Couche UI Affichage :
- ✓ Grille
- ✓ Score
- ✓ Boutons
- ✓ Menus
- ✓ Elle observe les états via :
- ✓ State Flow + collectas State ()
- ✓ Couche ViewModel

- ✓ Elle orchestre :
- ✓ Mouvements
- ✓ Sauvegarde
- ✓ Chargement
- ✓ Statistiques
- ✓ Modes spéciaux
- ✓ Classe principale :
- ✓ GameViewModel
- ✓ Couche logique métier
- ✓ Implémentée dans :
- ✓ GameEngine
- ✓ Responsable :
- ✓ Génération des tuiles
- ✓ Fusion
- ✓ Déplacements
- ✓ Conditions victoire/défaite
- ✓ Undo
- ✓ Couche persistance
- ✓ Deux systèmes utilisés :
- ✓ Data Store
- ✓ Room Data base

5. Fonctionnalités principales :

5.1 Mode classique :

Fonctionnalité centrale de l'application :

Le joueur déplace les tuiles dans quatre directions :

- Haut
- Bas
- Gauche
- Droite

Lorsque deux tuiles identiques se rencontrent :

Elles fusionnent et leur valeur double.

Une nouvelle tuile apparaît après chaque mouvement.

Objectif :

Atteindre 2048.

5.2 Gestion du score :

Le score augmente lors de chaque fusion :

Exemple :

2 + 2 = 4

Le score augmente de 4 points.

Le meilleur score est sauvegardé automatiquement.

5.3 Détection de fin de partie :

Deux cas :

Victoire

Lorsqu'une tuile 2048 apparaît

Défaite

Lorsqu'aucun mouvement n'est possible

5.4 Sauvegarde automatique :

L'état de la partie est sauvegardé automatiquement :

Grille

Score

Progression

Cela permet de reprendre la partie après fermeture de l'application.

5.5 Tableau des scores :

Un classement local est maintenu grâce à Room.

Fonctionnalités :

- Enregistrement des scores
- Tri automatique
- Affichage Top scores
- Suppression du classement

6. Fonctionnalités avancées :

6.1 Mode multi-joueurs

Deux joueurs jouent simultanément sur le même appareil.

Chaque joueur possède :

- Sa propre grille
- Son score
- Ses mouvements
- Deux modes disponibles :
- Mode libre
- Mode chronométré

À la fin :

Comparaison automatique des scores d'affichage du gagnant.

6.2 Mode défi quotidien :

Chaque jour :

Un défi unique est généré automatiquement.

Caractéristiques :

- Grille identique pour tous les joueurs.
- Objectif fixe
- Temps limité
- Score cible
- Le système utilise un speed basé sur la date.
- Cela garantit :
- Équité entre joueurs

6.3 Tutoriel interactif :

Le tutoriel guide l'utilisateur à travers :

- Déplacements
- Fusion
- Apparition de tuiles
- Objectifs du jeu

Il utilise une simulation animée de la grille.

6.4 Statistiques utilisateur :

L'application enregistre :

- ✓ Nombre de parties jouées
- ✓ Nombre de victoires
- ✓ Nombre de défaites
- ✓ Meilleur score

Ces statistiques sont visibles dans les paramètres.

6.5 Personnalisation graphique :

Plusieurs thèmes disponibles :

- Clair
- Sombre
- Coloré
- L'utilisateur peut modifier :
- Couleurs
- Animations
- Taille de grille

Tailles disponibles :

- 3×3
- 4×4
- 5×5
- 6×6

6.6 Gestion audio

Deux types de sons intégrés :

Sons de mouvement et musique de fond.

Ils peuvent être activés ou désactivés.

6.7 Partage du score :

L'utilisateur peut partager :

- Score
- Image PNG de la partie
- Fonctionne via :
- Intenta Android
- Compatible :
- Whasp
- Email
- Réseaux sociaux

7. Persistance des données :

Deux systèmes complémentaires sont utilisés.

7.1 Data Store :

Stocke :

- Paramètres utilisateur
- État de la partie
- Statistiques
- Avantage :
- Asynchrone
- Sécurisé
- Remplace SharedPreferences

7.2 Base Room :

Stocke :

- Classement des scores
- Structure :
- Score

- Date
- Taille de grille
- Permet :
- Tri rapide
- Requêtes efficaces

8. Gestion des états avec State Flow :

- State Flow permet :
- Mise à jour automatique de l'interface
- Lorsqu'un mouvement est effectué :
- GameEngine modifie la grille
- ViewModel reçoit la modification
- UI se met à jour automatiquement
- Sans rechargement manuel.

9. Interface utilisateur :

L'interface principale contient :

- Logo
- Score
- Best score
- Grille interactive
- Boutons directionnels
- Menu paramètres
- Menu scores
- Bouton partage
- Les animations utilisent :
- AnimateColorAsState ()
- Pour fluidifier les transitions.

10. Capture d'écran et partage :

Lorsqu'un utilisateur partage son score :

- L'écran est capturé
- Converti en image PNG
- Stocké temporairement
- Envoyé via File Provider Android

Cela garantit :

- Sécurité
- Compatibilité
- Performance

11. Optimisation des performances :

Plusieurs optimisations ont été appliquées :

- State Flow pour limiter recompositions inutiles
- Compose pour UI réactive
- Room pour requêtes rapides
- Data Store asynchrone
- Sound Pool pour sons courts

Résultat :

Application fluide même sur appareils modestes

12. Déploiement sur Google Play Store :

L'application a été :

- Compilée
- Signée
- Optimisée
- Puis publiée en version test interne Play Store
- Cela permet :
- Tests réels utilisateurs
- Détection bugs
- Validation compatibilité appareils

Cette étape représente un niveau avancé de finalisation du projet.

13. Phase actuelle du projet :

Le projet est actuellement en :

- Phase de test réel
- Objectifs :
- Valider stabilité
- Corriger bugs mineurs
- Améliorer UX
- Optimiser performances

Cette phase confirme que l'application est prête pour une diffusion publique.

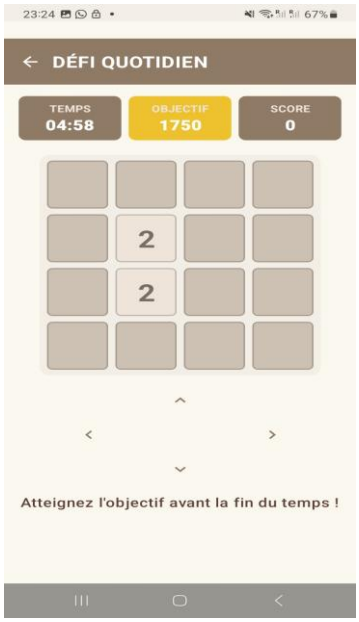
14. Qualité logicielle du projet :

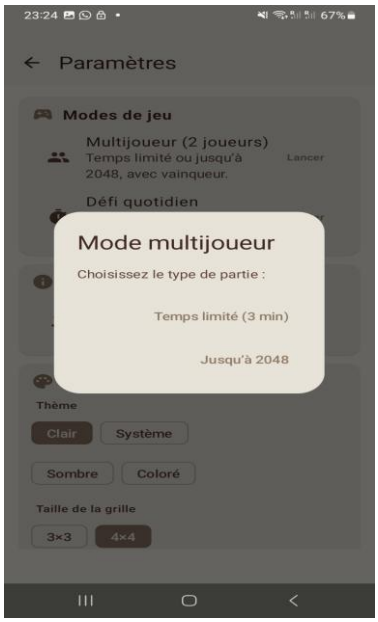
Le projet respecte plusieurs bonnes pratiques :

- Architecture MVVM
- Séparation logique/UI
- Persistance moderne
- Navigation Compose
- Gestion life cycle Android
- Code modulaire

Ce niveau correspond à une application professionnelle Android moderne.

15. Les captures :





16.1 Structure des paquets et rôle des fichiers

L'application suit une organisation classique Android :

- **com.jeu2048.app.game** — `GameEngine`, `GameState`, `Direction` : logique pure du jeu, sans dépendance Android UI.
- **com.jeu2048.app.ui** — `GameViewModel`, navigation, écrans ; orchestre les moteurs, les préférences et la base.
- **com.jeu2048.app.ui.components** — `GameGrid`, `TileCell`, `ScoreCard` : composants Compose réutilisables.
- **com.jeu2048.app.data** — DataStore (`GamePreferences`), Room (`ScoreEntity`, `ScoreDao`), autres préférences.

Cette séparation permet d'expliquer le fonctionnement de la **grille** en se limitant à `GameEngine` et `GameState`, puis de montrer comment l'UI ne fait qu'**observer** et réagir.

16.2 Représentation de la grille et état réactif

16.3 Tableau bidimensionnel

La grille est matérialisée par `Array<IntArray>`, c'est-à-dire un tableau de lignes, chaque ligne étant un `IntArray` de longueur `gridSize`. L'indice de ligne `r` et l'indice de colonne `c` donnent l'accès `grille[r][c]`. La valeur `0` signifie case vide ; toute autre valeur est une puissance de 2 affichée sur la tuile.

16.4 MutableStateFlow et StateFlow

Le moteur encapsule la grille dans un `MutableStateFlow` privé `_grid`, exposé en lecture seule via `val grid: StateFlow<Array<IntArray>>`. Chaque assignation `_grid.value = …` notifie les collecteurs. Dans Compose, `collectAsState()` transforme ce flux en état local : toute modification de grille déclenche une recomposition des composables qui lisent cette valeur.

Le même schéma s'applique au score, au meilleur score, aux drapeaux `hasWon`, `gameOver` et `lastMoveHadMerge` (utile pour choisir le son « fusion » ou « mouvement »).

16.5 Pourquoi cloner les tableaux

Les tableaux Kotlin sont mutables par référence. Lors d'une copie « superficielle », deux grilles partageraient les mêmes lignes. Le code utilise `.map { it.clone() }.toTypedArray()` pour dupliquer chaque ligne avant de stocker un snapshot (undo ou mise à jour), ce qui évite que l'historique ne soit corrigé par erreur lors des prochains calculs.

16.6 GameEngine : constructeur, nouvelle partie, graine, restauration

16.7 Contrainte sur la taille

Le bloc `init { require(gridSize in 3..6) }` garantit que seules les tailles prévues par les paramètres (3×3 à 6×6) sont utilisées ; une valeur incorrecte provoque une exception au plus tôt.

16.8 startNewGame et générateur aléatoire

`startNewGame()` alloue une grille neuve, remet les compteurs et drapeaux, vide l'historique d'undo, puis appelle deux fois `addRandomTile(g)`. Enfin il publie une copie clonée dans `_grid`. Le générateur `random` est par défaut `Random.Default` pour une partie classique.

16.9 startSeededGame pour le défi quotidien

```
fun startSeededGame(seed: Long) {
    random = Random(seed)
    startNewGame()
}
```

En remplaçant le générateur par `Random(seed)` avant `startNewGame()`, tous les tirages (positions des tuiles initiales, puis chaque nouvelle tuile après coup) deviennent **déterministes** pour une graine donnée. Le ViewModel utilise comme graine le nombre de jours depuis l'époque Unix pour la date du jour, ce qui aligne tous les joueurs sur la même « partie du jour ».

17. restoreGame et undo

`restoreGame` recharge grille et scores depuis le disque ; `undo()` dépile le dernier `GameSnapshot` (grille + score) pour revenir au coup précédent. La limite `maxUndo = 10` éviter une consommation mémoire illimitée.

17.1 Fonction move(direction) : déroulement détaillé

Voici la logique dans l'ordre d'exécution :

1. Si `gameOver`, retour `false`.
2. Copie profonde de `_grid` dans `current` et mémorisation du score courant.
3. Ajout d'un snapshot (grille + score) dans `moveHistory`, en supprimant le plus ancien si la liste dépasse 10 entrées.
4. Création d'un tableau `merged` de booléens par ligne : une ligne par rangée en mode horizontal, une ligne par colonne en mode vertical (voir section suivante).
5. Selon `Direction`, application de `mergeLine` à chaque ligne ou colonne transformée en problème 1D « vers la gauche ».
6. Agrégation : `changed` est vrai si au moins une ligne/colonne a signalé un mouvement ou une fusion (`moved`).
7. `lastMoveHadMerge` est mis à `true` si le score a augmenté (approximation « il y a eu au moins une fusion » dans ce modèle).
8. Si `!changed`, suppression du dernier snapshot et retour `false` — aucune nouvelle tuile n'est ajoutée.
9. Sinon : mise à jour du score et du meilleur score, `addRandomTile(current)`, publication de la grille, test victoire (tuile 2048), test défaite (`!canMove`).

17.2 Réduction des quatre directions

Pour **LEFT**, chaque `current[row]` est passé à `mergeLine` avec `merged[row]`.

Pour **RIGHT**, la ligne est inversée (`reversedArray()`), fusionnée avec un tableau `merged` neutre (les positions fusionnées suivent la ligne inversée), puis le résultat est réinversé pour réinjecter dans `current[row]`.

Pour **UP**, pour chaque colonne `col`, on construit `IntArray(gridSize) { current[it][col] }` : indices de ligne de 0 (haut) à `gridSize-1` (bas). Après fusion, on réécrit `current[row][col]` avec les valeurs de `mergedCol`.

Pour **DOWN**, on lit la colonne du bas vers le haut : `current[gridSize - 1 - it][col]` pour `it` de 0 à `gridSize-1`, ce qui place le « bas » de la colonne à gauche dans le tableau 1D passé à `mergeLine`. On réinjecte ensuite avec la boucle symétrique sur `row`.

17.3 Algorithme mergeLine : principe et trace sur un exemple

Signature : `mergeLine(line: IntArray, merged: BooleanArray): Triple<IntArray, Int, Boolean>`. Le tableau `merged` indique, pour chaque position du **résultat**, si une tuile à cet index a déjà été produite par une fusion dans ce même coup. Cela implémente la règle officielle : une tuile ne participe qu'à **une** fusion par mouvement dans une direction donnée.

Variables internes : `result` (ligne compactée à gauche), `write` (prochain index d'écriture), `scoreDelta` (somme des valeurs créées par fusion), `moved` (au moins une fusion dans cette implémentation lorsque le drapeau est positionné sur la branche fusion).

Exemple : ligne [2, 2, 2, 0], fusion vers la gauche

État initial : `write = 0`, `result` vide.

1. Premier `2` : pas de voisin à gauche → écriture en `result[0]`, puis `write = 1`.
2. Deuxième `2` : `result[write-1] == 2` et pas encore fusionné → fusion en `result[0] = 4`, `merged[0] = true`, `scoreDelta += 4` (le `write` n'avance pas sur une fusion).
3. Troisième `2` : `result[write-1]` pointe vers `result[0] = 4` ; `4 ≠ 2` → pas de fusion avec le 4 (interdit : une tuile déjà fusionnée ne fusionne pas deux fois dans le même coup). On écrit donc en `result[1] = 2` et `write = 2`.
4. Le `0` est ignoré.

Résultat : `[4, 2, 0, 0]`, conforme aux règles du 2048 : les deux premiers 2 fusionnent, le troisième 2 se place à côté du 4.

Code de référence (extrait)

```
private fun mergeLine(line: IntArray, merged: BooleanArray): Triple<IntArray, Int, Boolean>
{
    val result = IntArray(line.size)
    var write = 0
    var scoreDelta = 0
    var moved = false
    for (value in line) {
        if (value == 0) continue
        if (write > 0 && result[write - 1] == value && !merged[write - 1]) {
            result[write - 1] *= 2
            merged[write - 1] = true
            scoreDelta += result[write - 1]
            moved = true
        } else {
            result[write++] = value
        }
    }
    return Triple(result, scoreDelta, moved)
}
```

17.4 Nouvelle tuile et fin de partie

1- addRandomTile

On collecte la liste des paires `(r,c)` où `g[r][c]==0`. Si la liste est vide, on ne fait rien (grille pleine). Sinon on tire une case au hasard avec `empty.random(random)` et on assigne 2 (90 %) ou 4 (10 %). Le même objet `random` sert pour le mode classique et pour le mode graine.

2- canMove

Retourne vrai dès qu'il existe une case vide ou deux voisins horizontaux ou verticaux égaux et non nuls. Sinon aucun glissement ni fusion n'est possible : la partie est perdue. Ce test est appliqué après chaque coup valide.

17.5 GameState, sérialisation et DataStore

3-Modèle GameState

```
data class GameState(
    val gridSize: Int,
    val cells: List<Int>,
    val score: Int,
    val bestScore: Int,
    val hasWon: Boolean
) {
    fun toGrid(): Array<IntArray> {
        val g = Array(gridSize) { IntArray(gridSize) }
        cells.forEachIndexed { i, v ->
            val r = i / gridSize
            val c = i % gridSize
            if (r in 0 until gridSize && c in 0 until gridSize) g[r][c] = v
        }
        return g
    }
}
```

L'index `i` parcourt les lignes : les `gridSize` premiers entiers sont la ligne 0, etc. Cette représentation linéaire est simple à sérialiser en une chaîne unique.

4- GamePreferences

Le format stocké est du texte : `gridSize | liste de cellules séparées par des virgules | score | bestScore | 0 ou 1 pour hasWon.` Les fonctions `serialize` et `parseGameState` assurent la conversion bidirectionnelle. DataStore est utilisé de manière asynchrone (`edit` en coroutine) depuis le ViewModel.

17.6 getStateForSave dans GameEngine

```
fun getStateForSave(): GameState = GameState(
    gridSize = gridSize,
    cells = _grid.value.flatMap { it.toList() },
    score = _score.value,
    bestScore = _bestScore.value,
    hasWon = _hasWon.value
)
```

`flatMap { it.toList() }` aplatit chaque ligne dans l'ordre des lignes, ce qui correspond exactement à `toGrid()` à la relecture.

18. GameViewModel : orchestration

1-Initialisation

Au démarrage, le ViewModel lit la taille de grille dans les paramètres, construit un `GameEngine(size)`, puis soit restaure `saved` si la sauvegarde correspond à la taille et au nombre de cellules attendu, soit appelle `startNewGame()`.

2-move et sauvegarde

`move(direction)` appelle `engine.move(direction)`. Si le booléen retourné est vrai, le ViewModel joue un son selon `lastMoveHadMerge`, puis `saveState()` qui persiste via `gamePrefs.saveGame(engine.getStateForSave())`. En cas de `gameOver`, statistiques et insertion Room peuvent être déclenchées.

3- Changement de taille de grille

`setGridSize` recrée un nouveau moteur avec la nouvelle dimension et lance une partie neuve : la logique de jeu reste identique, seul `gridSize` change dans `GameEngine`.

19. Base Room : classement local

L'entité `ScoreEntity` contient un identifiant auto-généré, le score, la taille de grille et un horodatage. Le DAO expose `getTopScores(limit)` comme un `Flow` trié par score décroissant, et `insert` / `deleteAll`. L'écran des scores observe ce flux pour afficher le top sans requête manuelle après chaque insertion.

1-GameScreen, GameGrid et interaction

2-Collecte des flux dans GameScreen

`GameScreen` reçoit une référence optionnelle vers `GameEngine`. Pour la grille, le pattern utilisé est `engine?.grid?.collectAsState(initial = …)` : dès que le moteur publie une nouvelle grille, l'UI se met à jour. Les scores et drapeaux `hasWon` / `gameOver` suivent le même principe. Des `LaunchedEffect` déclenchent les callbacks `onWin` et `onGameOver` pour sons ou boîtes de dialogue.

3-GameGrid et TileCell

`GameGrid` itère sur `r` et `c`, lit `grid[r][c]` et compose une `TileCell` par case. Les surbrillances de ligne/colonne servent au retour visuel après un geste ou un bouton directionnel. `TileCell` utilise `animateColorAsState` pour interpoler la couleur de fond selon la valeur et le thème ; la taille de police est proportionnelle à la contrainte disponible pour garder des chiffres lisibles sur 6×6.

20. Modes multijoueur et défi quotidien

1-Multijoueur

Deux instances `GameEngine` distinctes (`player1Engine`, `player2Engine`) partagent la même règle de déplacement. Les méthodes `movePlayer1` et `movePlayer2` appellent `move` sur le bon moteur.

La fin peut être déclenchée par le chronomètre, par l'atteinte de 2048 en mode non chronométré, ou par l'impossibilité de jouer pour les deux grilles. Le gagnant est déduit de la comparaison des scores.

2-Défi quotidien

La graine `LocalDate.now().toEpochDay()` fixe la partie et intervient aussi dans le calcul d'un objectif de score (`challengeTargetFor`) tenant compte de la taille de grille. Un compte à rebours est lancé en parallèle ; la réussite compare le score courant à l'objectif avant expiration du temps ou avant game over.

16. Conclusion :

Ce projet de développement d'une application mobile Android du jeu 2048 a permis de mettre en pratique de manière concrète les concepts fondamentaux du développement mobile moderne. À travers la conception, l'implémentation et l'amélioration progressive de application, plusieurs compétences techniques ont été mobilisées, notamment en programmation avec Kotlin, en création d'interfaces avec Jetpack Compose et en structuration du code selon l'architecture MVVM.

La réalisation de Ce projet a également permis de mieux comprendre les enjeux liés à la gestion des interactions utilisateur, à la manipulation des données en temps réel ET à la persistance de l'état d'une application. L'intégration de fonctionnalités telles que la gestion du score, la sauvegarde automatique, les paramètres ou encore les animations a contribué à enrichir l'expérience utilisateur ET à rendre l'application plus complète ET interactive.

Au-delà de l'aspect technique, Ce travail a mis en évidence l'importance de l'organisation du code, de la modularité ET du respect des bonnes pratiques de développement. Ces éléments sont essentiels pour garantir la maintenabilité, l'évolutivité ET la qualité globale d'une application.

Enfin, Ce projet ouvre la voie à plusieurs perspectives d'amélioration, telles que l'ajout d'un mode multijoueur, l'intégration d'un classement en ligne ou encore l'optimisation des performances. IL constitue ainsi une base solide pour des développements futurs ET une étape importante dans l'apprentissage du développement mobile.

Ce projet a permis de concevoir une application Android complète basée sur le jeu 2048 intégrant des fonctionnalités avancées dépassant largement les exigences initiales.

- ✓ L'intégration :
- ✓ Multi-joueurs
- ✓ Défi quotidien
- ✓ Statistiques
- ✓ Partage d'image
- ✓ Personnalisation
- ✓ Base Room
- ✓ Data Store
- ✓ Déploiement Play Store

Démontre une maîtrise avancée du développement mobile Android moderne.

L’application est aujourd’hui fonctionnelle, testée et prête à évoluer vers une version publique stable.